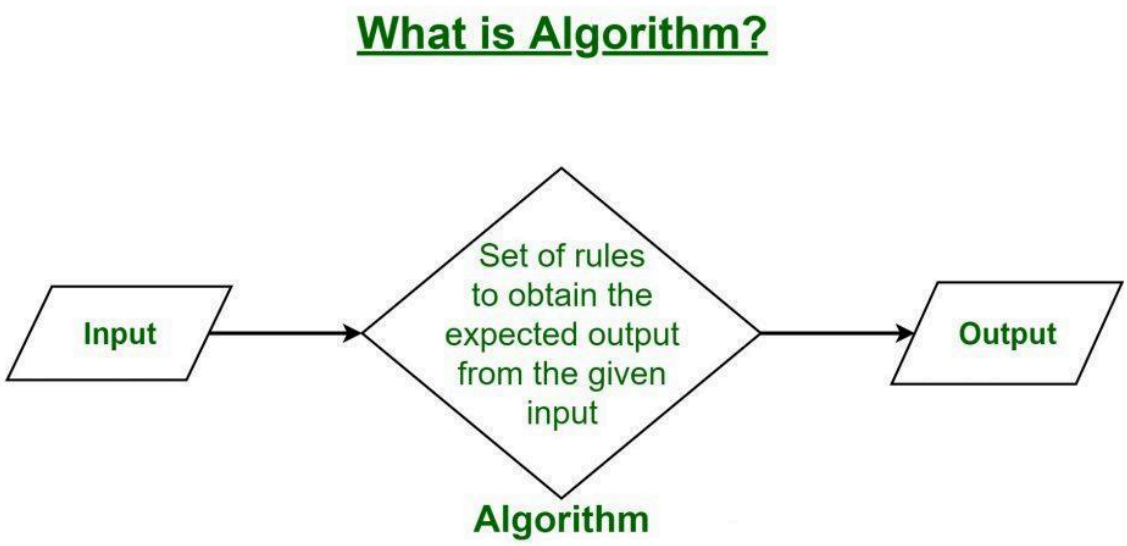
ALGORITHM

An algorithm is a set of instructions designed to perform a specific task. This can be a simple process, such as multiplying two numbers, or a complex operation, such as playing a compressed video file. Search engines use proprietary algorithms to display the most relevant results from their search index for specific queries.

In many cases, there are multiple ways to perform a specific operation within a software program. Therefore, programmers usually seek to create the most efficient algorithms possible.



EXAMPLES:

WRITE AN ALGORITHM TO ADD TWO NUMBERS

```
Step 1: Start

Step 2: Declare variables num1, num2 and sum.

Step 3: Read values num1 and num2.

Step 4: Add num1 and num2 and assign the result to sum.

        sum←num1+num2

Step 5: Display sum

Step 6: Stop
```

WRITE AN ALGORITHM TO FIND FACTORIAL OF A NUMBER ENTERED BY USER

```
Step 1: Start

Step 2: Declare variables n, factorial and i.

Step 3: Initialize variables

          factorial ← 1

i ← 1

Step 4: Read value of n

Step 5: Repeat the steps until i = n

     5.1: factorial ← factorial*i

     5.2: i ← i+1

Step 6: Display factorial

Step 7: Stop
```

FLOWCHART

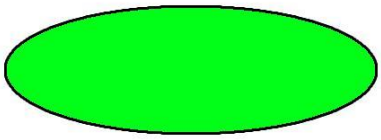
What is a Flowchart?

Flowchart is a graphical representation of an algorithm. Programmers often use it as a program-planning tool to solve a problem. It makes use of symbols which are connected among them to indicate the flow of information and processing.

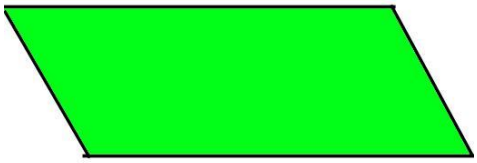
The process of drawing a flowchart for an algorithm is known as “flowcharting”.

Basic Symbols used in Flowchart Designs

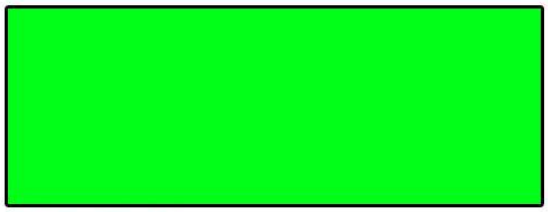
1. **Terminal:** The oval symbol indicates Start, Stop and Halt in a program’s logic flow. A pause/halt is generally used in a program logic under some error conditions. Terminal is the first and last symbols in the flowchart.



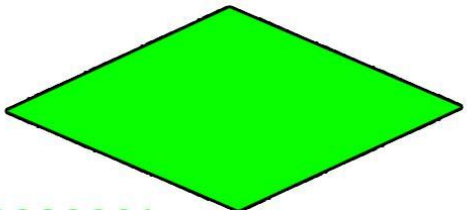
- **Input/Output:** A parallelogram denotes any function of input/output type. Program instructions that take input from input devices and display output on output devices are indicated with parallelogram in a flowchart.



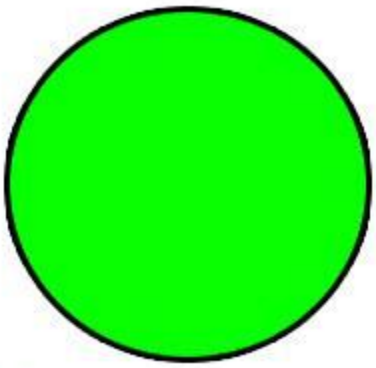
- **Processing:** A box represents arithmetic instructions. All arithmetic processes such as adding, subtracting, multiplication and division are indicated by action or process symbol.



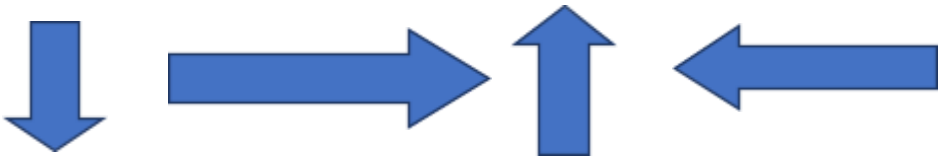
- **Decision** Diamond symbol represents a decision point. Decision based operations such as yes/no question or true/false are indicated by diamond in flowchart.



- **Connectors:** Whenever flowchart becomes complex or it spreads over more than one page, it is useful to use connectors to avoid any confusions. It is represented by a circle.



- **Flow lines:** Flow lines indicate the exact sequence in which instructions are executed. Arrows represent the direction of flow of control and relationship among different symbols of flowchart.



Advantages of Flowchart:

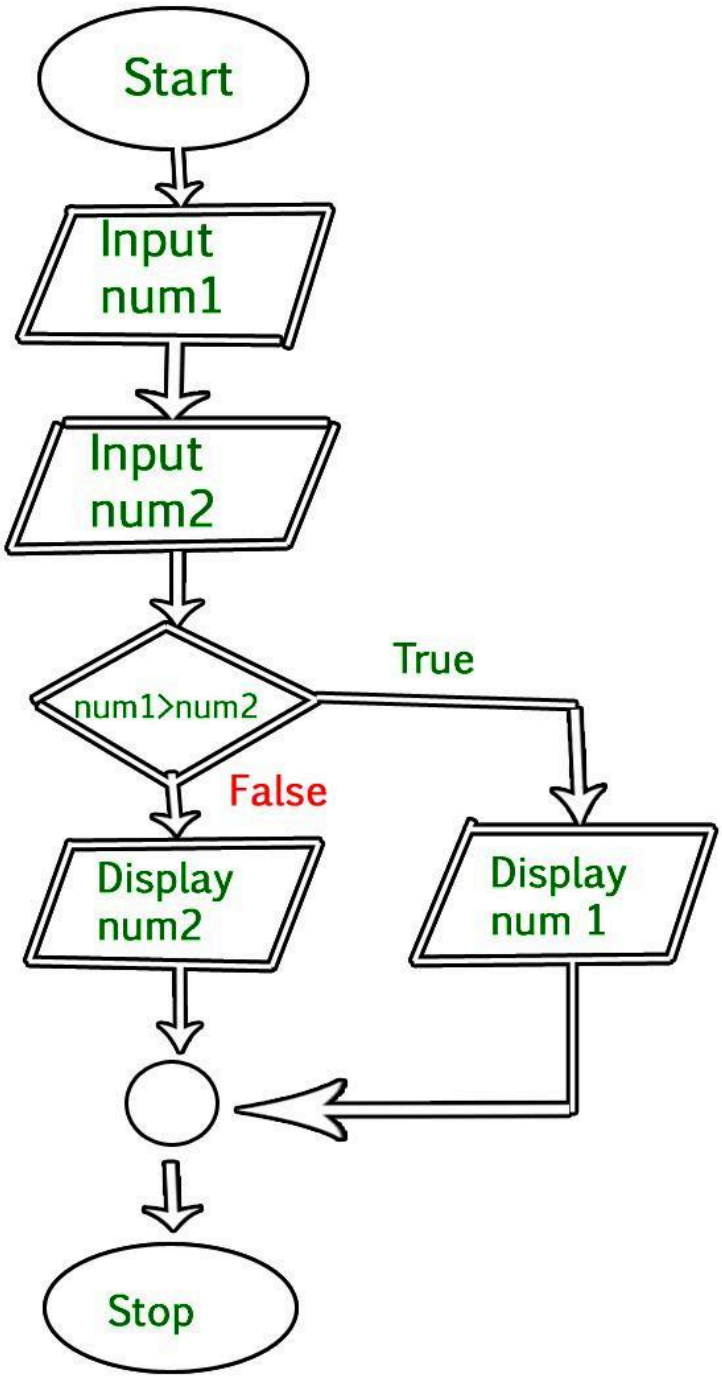
- Flowcharts are better way of communicating the logic of system.
- Flowcharts act as a guide for blueprint during program designed.
- Flowcharts helps in debugging process.
- With the help of flowcharts programs can be easily analyzed.
- It provides better documentation.

- Flowcharts serve as a good proper documentation.

Disadvantages of Flowchart:

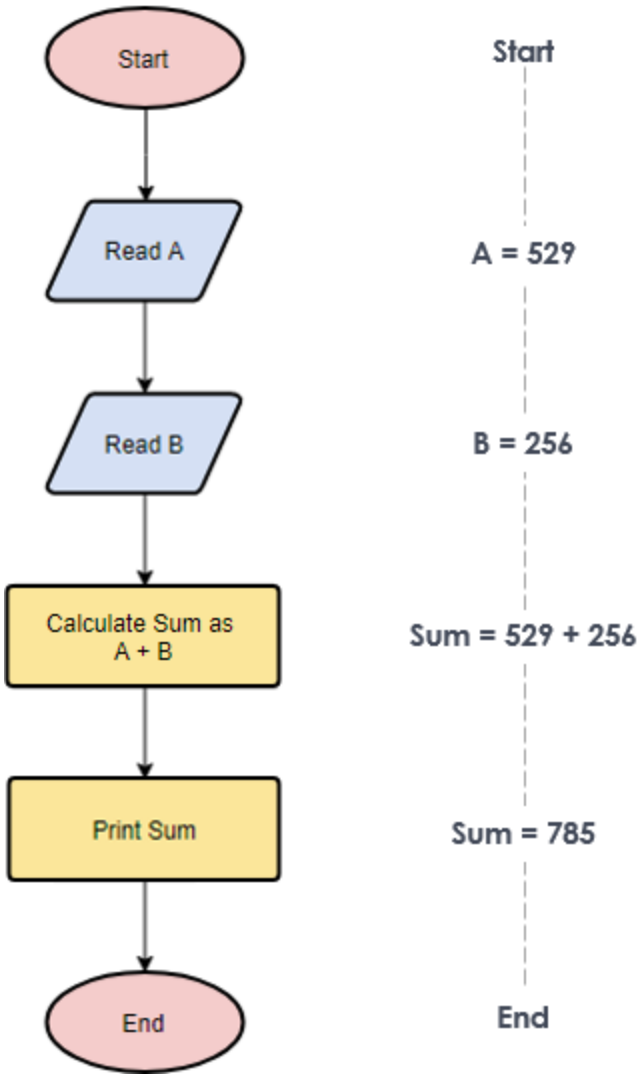
- It is difficult to draw flowchart for large and complex programs.
- In this their is no standard to determine the amount of detail.
- Difficult to reproduce the flowcharts.
- It is very difficult to modify the Flowchart.

Example : Draw a flowchart to input two numbers from user and display the largest of two numbers



FLOWCHART TO ADD TWO NUMBERS

Find the sum of 529 and 256



PSEUDOCODE

Pseudo code is a term which is often used in programming and algorithm based fields. It is a methodology that allows the programmer to represent the implementation of an algorithm. Simply, we can say that it's the cooked up representation of an algorithm. Often at times, algorithms are represented with the help of pseudo codes as they can be interpreted by programmers no matter what their programming background or knowledge is. Pseudo code, as the name suggests, is a false code or a representation

of code which can be understood by even a layman with some school level programming knowledge.

Algorithm: It's an organized logical sequence of the actions or the approach towards a particular problem. A programmer implements an algorithm to solve a problem. Algorithms are expressed using natural verbal but somewhat technical annotations.

Pseudo code: It's simply an implementation of an algorithm in the form of annotations and informative text written in plain English. It has no syntax like any of the programming language and thus can't be compiled or interpreted by the computer.

Advantages of Pseudocode

- Improves the readability of any approach. It's one of the best approaches to start implementation of an algorithm.
- Acts as a bridge between the program and the algorithm or flowchart. Also works as a rough documentation, so the program of one developer can be understood easily when a pseudo code is written out. In industries, the approach of documentation is essential. And that's where a pseudo-code proves vital.
- The main goal of a pseudo code is to explain what exactly each line of a program should do, hence making the code construction phase easier for the programmer.

How to write a Pseudo-code?

1. Arrange the sequence of tasks and write the pseudocode accordingly.
2. Start with the statement of a pseudo code which establishes the main goal or the aim.

   Example:

   ```
   This program will allow the user to check

   the number whether it's even or odd.
   ```

3. The way the if-else, for, while loops are indented in a program, indent the statements likewise, as it helps to comprehend the decision control and execution mechanism. They also improve the readability to a great extent.
4. Use appropriate naming conventions. The human tendency follows the approach to follow what we see. If a programmer goes through a pseudo code, his approach will be the same as per it, so the naming must be simple and distinct.
5. Use appropriate sentence casings, such as CamelCase for methods, upper case for constants and lower case for variables.
6. Elaborate everything which is going to happen in the actual code. Don't make the pseudo code abstract.
7. Use standard programming structures such as 'if-then', 'for', 'while', 'cases' the way we use it in programming.
8. Check whether all the sections of a pseudo code is complete, finite and clear to understand and comprehend.
9. Don't write the pseudo code in a complete programmatic manner. It is necessary to be simple to understand even for a layman or client, hence don't incorporate too many technical terms.

EXAMPLES

WRITE A PSEUDOCODE TO FIND LARGEST OF THREE NUMBERS

1. READ the values of 3 numbers A, B and C

2. IF the value of A>that of B THEN

3. IF the value of A>that of C THEN

4. IF the value of B>that of C THEN

5. STOP processing

WRITE A PSEUDOCODE TO FIND SUM OF n NATURAL NUMBERS

- int i, num, s=0
- input num
- i=0
- do
  s=s+i
  i=i+1
  till the value of i<=num
- print *s* as the sum of numbers.

WRITE A PSEUDOCODE TO FIND FACTORIAL OF A NUMBER

- Pseudocode for Factorial of a number :
- Step 1: Declare N and F as integer variable.
- Step 2: Initialize F=1.
- Step 2: Enter the value of N.
- Step 3: Check whether N>0, if not then F=1.
- Step 4: If yes then, F=F*N
- Step 5: Decrease the value of N by 1 .
- Step 6: Repeat step 4 and 5 until N=0.
- Step 7: Now print the value of F.
- The value of F will be the factorial of N(number).